



UNIVERSITE HASSAN II DE CASABLANCA
FACULTE DES SCIENCES BEN M'SICK

Documentation PFE - PDF 05/10

MongoDB + NoSQL

Base documentaire flexible + 6 collections + Motor async

Realise par :

AKRAM BELMOUSSA - Backend & Integration NLP

ZAKARIA - Frontend Angular & UX/UI

NOUHAILA - Base de donnees & Contenu FAQ

MAY 2026

Sommaire

Chapitre 1 - C'est quoi NoSQL ?	3
Chapitre 2 - SQL vs NoSQL	6
Chapitre 3 - C'est quoi MongoDB ?	9
Chapitre 4 - Documents BSON	12
Chapitre 5 - Les 6 collections de fsbm_reviews	15
Chapitre 6 - JSON Schema validation	20
Chapitre 7 - Index dans MongoDB	23
Chapitre 8 - Requetes MongoDB	25
Chapitre 9 - Motor (driver async Python)	29
Chapitre 10 - Quand utiliser MongoDB vs MySQL	32
Chapitre 11 - Conclusion	35

Chapitre 1 - C'est quoi NoSQL ?

1.1 Definition

NoSQL signifie '**Not Only SQL**' (pas seulement SQL). C'est une famille de bases de donnees qui ne suivent **pas le modele relationnel** tabulaire.

■ **ANALOGIE** : SQL = un **classeur Excel** avec lignes/colonnes rigides. NoSQL = un **tiroir** ou tu mets des dossiers de toutes les formes. Chaque dossier (document) peut avoir une structure differente.

1.2 Les 4 categories de NoSQL

Type	Exemples	Use case
Documentaire	MongoDB, CouchDB	Reviews, logs, content
Cle-valeur	Redis, DynamoDB	Cache, sessions
Colonnes	Cassandra, HBase	Analytics big data
Graphes	Neo4j, ArangoDB	Reseaux sociaux

Notre projet utilise **MongoDB** (documentaire). C'est le NoSQL le plus populaire.

1.3 Histoire

Le terme 'NoSQL' a explose en 2009 quand Google, Amazon, Facebook avaient besoin de gerer des volumes massifs de donnees flexibles que les SGBD relationnels classiques geraient mal. MongoDB est sorti en 2009 par 10gen (devenu MongoDB Inc.).

1.4 Pourquoi NoSQL ?

- + **Schema flexible** : pas besoin de migrer pour ajouter un champ
- + **Scaling horizontal** : facile de repartir sur plusieurs serveurs (sharding)
- + **Performance** sur de gros volumes (millions de docs)
- + **JSON natif** : pas besoin de mapper objet <-> table
- + **Developpement rapide** : ideal pour MVP et prototypes

Chapitre 2 - SQL vs NoSQL

2.1 Comparaison detaillee

Critere	SQL (MySQL)	NoSQL (MongoDB)
Modele	Tables avec lignes	Collections de documents
Schema	Fixe et strict	Flexible (peut etre valide)
Relations	JOINS natifs	Embedding ou references
Transactions	ACID complet	ACID depuis v4
Langage	SQL standardise	API JavaScript-like
Scaling	Vertical (gros serveur)	Horizontal (cluster)
Apprentissage	Plus connu	Plus recent
Cas ideal	Donnees structurees liees	Donnees variables, gros volume
Exemples	Banque, ERP, comptabilite	CMS, logs, reviews, IoT
Dans FSBM	Etudiants, profs, notes	Reviews, logs, sentiments

2.2 Quand choisir l'un ou l'autre ?

Choisis SQL si :

- Tes donnees sont fortement liees (relations N:N complexes)
- Tu as besoin de transactions ACID strictes (banque, comptabilite)
- Ton schema est stable et bien defini
- Tu utilises beaucoup de JOINS et d'aggregations
- Ton volume est petit a moyen (< 100M lignes)

Choisis NoSQL si :

- Tes donnees sont variables / evolutives
- Tu as enormement de donnees (terabytes)
- Tu fais surtout des lookups simples (par cle)
- Tu prefiers une structure hierarchique JSON
- Tu veux scaler horizontalement facilement

[★] Dans notre PFE, on utilise **les DEUX**. C'est ce qu'on appelle '**polyglot persistence**'. Chaque type de donnees va dans la base qui lui convient. MySQL pour les donnees academiques structurees, MongoDB pour les feedbacks flexibles.

Chapitre 3 - C'est quoi MongoDB ?

3.1 MongoDB en bref

MongoDB est le SGBD documentaire le plus populaire au monde. Il stocke des **documents JSON-like** dans des **collections**.

3.2 Vocabulaire

SQL	MongoDB
Database	Database
Table	Collection
Row	Document
Column	Field
Primary Key	_id (ObjectId auto)
JOIN	\$lookup (rarement utilise)
SQL query	Query API (JSON)

3.3 Exemple de document

```
// Une review dans la collection 'reviews'
{
  "_id": ObjectId("6553..."),
  "user_id": 12345,
  "username": "Yassine B.",
  "rating": 5,
  "title": "Tres utile pour les questions de scolarite !",
  "content": "Le chatbot m'a beaucoup aide...",
  "category": "CHATBOT",
  "sentiment": "POSITIVE",
  "sentiment_score": 0.92,
  "tags": ["scolarite", "rapide", "clair"],
  "is_verified": true,
  "helpful_count": 23,
  "created_at": ISODate("2026-04-15T10:30:00Z")
}
```

Note les types varies : nombres, strings, booleens, tableaux, dates, sous-documents. Tout en un seul document.

3.4 Outils MongoDB

- **mongosh** - shell ligne de commande (comme mysql CLI)
- **MongoDB Compass** - GUI desktop officielle
- **Studio 3T** - GUI avantee (payante)

- **MongoDB Atlas** - service cloud manage (free tier 512MB)

Chapitre 4 - Documents BSON

4.1 JSON vs BSON

MongoDB ne stocke pas du JSON pur, mais du **BSON = Binary JSON**. C'est un format binaire compact qui supporte plus de types que JSON :

Type BSON	JSON equivalent	Note
String	String	UTF-8
Int32 / Int64	Number	Distinction int/float
Double	Number	Floats
Boolean	Boolean	true/false
Date	String ISO	Stocke en ms epoch
ObjectId	-	ID unique 12 bytes
Array	Array	Liste hetero
Object	Object	Sous-document
Binary	-	Donnees binaires
Null	null	Absence

4.2 _id et ObjectId

Chaque document a un champ `_id` unique (genere automatiquement). C'est un **ObjectId** de 12 bytes contenant : timestamp + machine + counter.

```
// Exemple : ObjectId("655398754d6e2f3c1a0b8c92")
// |-----||----||-----|
// timestamp machine pid counter

// Tu peux le forcer :
db.users.insertOne({ _id: "karim_alaoui", name: "Karim" });

// Sinon MongoDB en genere un :
db.users.insertOne({ name: "Fatima" });
// -> _id: ObjectId("...") auto
```

4.3 Embedding vs References

Pour modeliser des relations, MongoDB offre 2 strategies :

Embedding (imbriquer)

```
{
  "_id": ...,
  "title": "Avis sur le chatbot",
  "author": {
    "name": "Karim",
```

```
    "email": "karim@etu.fsbm.ma"
  },
  "comments": [
    { "user": "Salma", "text": "D'accord !" },
    { "user": "Hamza", "text": "Aussi mon avis." }
  ]
}
```

References (separation)

```
// Collection 'reviews'
{ "_id": ObjectId("a1"), "title": "...", "author_id": ObjectId("u1") }

// Collection 'users' separee
{ "_id": ObjectId("u1"), "name": "Karim", "email": "..." }
```

[★] Regle : embed si on lit toujours ensemble, refer si volumes gros ou modifs frequentes.
Pour notre FSBM : commentaires **embed** dans reviews (toujours lus ensemble), user_id **refer** (utilisateur evolue independamment).

Chapitre 5 - Les 6 collections de fsbm_reviews

- 1. reviews.** Avis textuels détaillés des utilisateurs sur le chatbot ou la plateforme. Champs : user_id, rating (1-5), title, content, category, sentiment, tags, helpful_count, created_at.
- 2. chatbot_feedback.** Feedback rapide (pouce haut/bas) lié a une conversation chatbot spécifique. Champs : conversation_id, session_id, message_text, bot_response, intent_detected, confidence, is_helpful, comment.
- 3. conversations.** Log complet d'une session de conversation chatbot. Embeddes : array de messages (role, text, timestamp, intent, confidence). Plus metadata : duration, device_type.
- 4. sentiment_analysis.** Resultats d'analyse de sentiment (NLP). source_type (REVIEW/FEEDBACK/MESSAGE), score (-1 a 1), emotions (joy, anger, sadness, surprise, fear), language.
- 5. usage_logs.** Tracking anonyme : page_view, chat_start, message_sent, search, login, feedback. event_type + session_id + duration_ms + timestamp.
- 6. suggestions.** Suggestions d'amelioration soumises par les utilisateurs. title, description, category, status (OUVERT/EN_COURS/ETUDE/FAIT), votes, author.

5.1 Structure complete : reviews

```
{
  _id: ObjectId("..."),
  user_id: 12345,
  username: "Yassine B.",
  rating: 5,                // int 1-5
  title: "Tres utile !",
  content: "Le chatbot...", // max 2000 chars
  category: "CHATBOT",      // enum : CHATBOT|INTERFACE|PERFORMANCE|CONTENT|AUTRE
  sentiment: "POSITIVE",    // enum : POSITIVE|NEUTRAL|NEGATIVE
  sentiment_score: 0.92,    // float
  tags: ["scolarite", "rapide"],
  is_verified: true,
  helpful_count: 23,
  created_at: ISODate("2026-04-15T10:30:00Z"),
  updated_at: ISODate("2026-04-15T10:30:00Z")
}
```

Chapitre 6 - JSON Schema validation

6.1 Le risque de la flexibilite

MongoDB est **trop flexible** par default. Sans validation, on peut inserer n'importe quoi (un rating string au lieu de int, un email mal formate, etc.).

6.2 Solution : JSON Schema validator

```
db.createCollection("reviews", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["user_id", "rating", "created_at"],
      properties: {
        user_id: { bsonType: ["int", "long", "string"] },
        username: { bsonType: "string" },
        rating: { bsonType: "int", minimum: 1, maximum: 5 },
        title: { bsonType: "string", maxLength: 200 },
        content: { bsonType: "string", maxLength: 2000 },
        category: {
          enum: ["CHATBOT", "INTERFACE", "PERFORMANCE", "CONTENT", "AUTRE"]
        },
        sentiment: {
          enum: ["POSITIVE", "NEUTRAL", "NEGATIVE"]
        },
        sentiment_score: { bsonType: "double" },
        tags: { bsonType: "array", items: { bsonType: "string" } },
        is_verified: { bsonType: "bool" },
        helpful_count: { bsonType: "int" },
        created_at: { bsonType: "date" }
      }
    }
  }
});
```

Si on tente d'insérer un document qui ne respecte pas le schema, MongoDB rejette :

```
db.reviews.insertOne({
  user_id: 1,
  rating: "5",      // ERREUR : devrait etre int
  created_at: new Date()
});
// Document failed validation
```

Chapitre 7 - Index dans MongoDB

7.1 Creation d'index

```
// Index simple
db.reviews.createIndex({ user_id: 1 });    // 1 = ascendant, -1 = descendant

// Index compose
db.reviews.createIndex({ rating: 1, created_at: -1 });

// Index unique
db.users.createIndex({ email: 1 }, { unique: true });

// Index TEXT (recherche full-text)
db.reviews.createIndex({ content: "text", title: "text" });

// Liste des index
db.reviews.getIndexes();
```

7.2 Index dans notre projet

```
// Collection reviews
db.reviews.createIndex({ user_id: 1 });
db.reviews.createIndex({ rating: 1, created_at: -1 });
db.reviews.createIndex({ category: 1 });
db.reviews.createIndex({ sentiment: 1 });
db.reviews.createIndex({ content: "text", title: "text" });

// Collection chatbot_feedback
db.chatbot_feedback.createIndex({ conversation_id: 1 });
db.chatbot_feedback.createIndex({ session_id: 1 });
db.chatbot_feedback.createIndex({ is_helpful: 1, created_at: -1 });
```

Chapitre 8 - Requetes MongoDB

8.1 find()

```
// Tout
db.reviews.find();

// Avec filtre
db.reviews.find({ rating: 5 });

// Operateurs : $gt, $lt, $gte, $lte
db.reviews.find({ rating: { $gte: 4 } });

// AND implicite
db.reviews.find({ rating: 5, sentiment: "POSITIVE" });

// OR
db.reviews.find({
  $or: [
    { rating: 5 },
    { sentiment: "POSITIVE" }
  ]
});

// IN
db.reviews.find({ category: { $in: ["CHATBOT", "INTERFACE"] } });

// Regex (recherche partielle)
db.reviews.find({ title: { $regex: /utile/i } });

// Tri + limite
db.reviews.find()
  .sort({ created_at: -1 })
  .limit(10);

// Projection (selection de champs)
db.reviews.find({}, { title: 1, rating: 1, _id: 0 });
```

8.2 insertOne / insertMany

```
db.reviews.insertOne({
  user_id: 1,
  rating: 5,
  title: "Super !",
  created_at: new Date()
});

db.reviews.insertMany([
  { user_id: 2, rating: 4, ... },
  { user_id: 3, rating: 5, ... }
]);
```

8.3 updateOne / updateMany

```
// Modifier un document
db.reviews.updateOne(
```

```
{ _id: ObjectId("...") },
{ $set: { rating: 4, sentiment: "NEUTRAL" } }
);

// Incrementer
db.reviews.updateOne(
  { _id: ObjectId("...") },
  { $inc: { helpful_count: 1 } }
);

// Ajouter a un array
db.reviews.updateOne(
  { _id: ObjectId("...") },
  { $push: { tags: "recommande" } }
);
```

8.4 Aggregation Pipeline

Pour des requetes complexes (group by, joins, calculs) :

```
// Compter les reviews par categorie
db.reviews.aggregate([
  { $group: {
    _id: "$category",
    count: { $sum: 1 },
    avg_rating: { $avg: "$rating" }
  }},
  { $sort: { count: -1 } }
]);

// Resultat :
// [
//   { _id: "CHATBOT", count: 42, avg_rating: 4.5 },
//   { _id: "INTERFACE", count: 18, avg_rating: 4.8 },
// ]
```

Chapitre 9 - Motor (driver async Python)

9.1 Pourquoi Motor ?

Le driver officiel **pymongo** est synchrone. Pour FastAPI async, on utilise **Motor** (driver async officiel MongoDB).

9.2 Installation et setup

```
pip install motor

# review_service/app/db.py
from motor.motor_asyncio import AsyncIOMotorClient

MONGO_URL = "mongodb://localhost:27017"
client = AsyncIOMotorClient(MONGO_URL)
db = client.fsbm_reviews

# Acces aux collections
reviews_col = db.reviews
feedback_col = db.chatbot_feedback
```

9.3 CRUD asynchrone avec Motor

```
# INSERT
result = await reviews_col.insert_one({
    "user_id": 1,
    "rating": 5,
    "title": "Super !",
    "created_at": datetime.utcnow()
})
print(result.inserted_id)

# FIND ONE
review = await reviews_col.find_one({"_id": ObjectId("...")})

# FIND MANY (cursor async)
cursor = reviews_col.find({"rating": {"$gte": 4}}).sort("created_at", -1).limit(10)
reviews = await cursor.to_list(length=10)

# UPDATE
await reviews_col.update_one(
    {"_id": ObjectId("...")},
    {"$inc": {"helpful_count": 1}}
)

# DELETE
await reviews_col.delete_one({"_id": ObjectId("...")})
```

9.4 Integration FastAPI

```
@router.post("/api/reviews")
async def create_review(review: ReviewCreate):
    doc = review.model_dump()
```

```
doc["created_at"] = datetime.utcnow()
result = await reviews_col.insert_one(doc)
return {"id": str(result.inserted_id)}

@router.get("/api/reviews")
async def list_reviews(limit: int = 20):
    cursor = reviews_col.find().sort("created_at", -1).limit(limit)
    reviews = await cursor.to_list(length=limit)
    # Conversion ObjectId -> str pour JSON
    for r in reviews:
        r["_id"] = str(r["_id"])
    return reviews
```

Chapitre 10 - Quand utiliser MongoDB vs MySQL

10.1 Repartition dans notre projet

Type de donnees	BDD choisie	Justification
Etudiants (CNE, nom, filiere)	MySQL	Schema fixe, relations strictes
Filieres + departements	MySQL	Relations N:N, integrite
Notes etudiants	MySQL	Calculs, transactions
Modules + emploi du temps	MySQL	Relations complexes
Reviews textuelles	MongoDB	Schema variable, full-text search
Sentiment analyses	MongoDB	Structure JSON imbriquee
Logs d'usage	MongoDB	Volume massif, ecritures
Suggestions ameliorations	MongoDB	Champs evoluent souvent
Conversations chatbot	MySQL+Mongo	MySQL pour historique strict, Mongo pour logs riches

10.2 Erreurs classiques

Erreur 1 : tout en MongoDB pour 'simplifier'

Tu perds l'integrite referentielle, les transactions ACID, les JOINS efficaces. Pour gerer un cursus academique avec notes/modules/profs lies, MySQL est BIEN MEILLEUR.

Erreur 2 : tout en SQL et stocker du JSON dans BLOBs

Pour des reviews flexibles, c'est faisable en MySQL (champ JSON) mais bien moins performant pour les recherches. MongoDB est natif pour ce cas.

10.3 Bilan pour le jury

■ Polyglot persistence = utiliser plusieurs BDD selon le type de donnees. C'est une pratique **industrielle** (Amazon, Netflix, Spotify). Demontrer qu'on la maitrise est un grand plus en soutenance.

Chapitre 11 - Conclusion

11.1 Recap des concepts

- + NoSQL = 'Not Only SQL' = familles non-relationnelles
- + 4 categories : Documentaire (MongoDB), Cle-valeur (Redis), Colonnes (Cassandra), Graphes (Neo4j)
- + MongoDB = documentaire, JSON-like, schema flexible
- + BSON = JSON binaire avec plus de types
- + Collections = tables, Documents = rows
- + _id ObjectId = cle primaire auto
- + Embedding vs References pour modeliser
- + JSON Schema validator pour contraintes
- + Motor = driver async Python pour FastAPI
- + Polyglot persistence = MySQL + MongoDB ensemble

11.2 Pour aller plus loin

- PDF 04 - pour bien comprendre MySQL et SQLAlchemy
- PDF 06 - pour le NLP et l'IA (utilise MongoDB pour logs en Phase 2)
- PDF 08 - pour la securite MongoDB